20.11.25

Stream Processing (Message Brokers and Kafka) Part 1

University of Vienna / Faculty of Computer Science

Andrey Nagiyev BSc MSc andrey.nagiyev@univie.ac.at

OUTLINE

- Problems of microservice Architecture:
  - Real life challenges
- Processing:
  - Batch processing
  - Stream processing
  - Lambda architecture
- Event-driven architecture
- Brokers
- Data Liberation

Learning Outcomes

Understanding the following terms.

For the Lecture 1 (more theoretical questions):

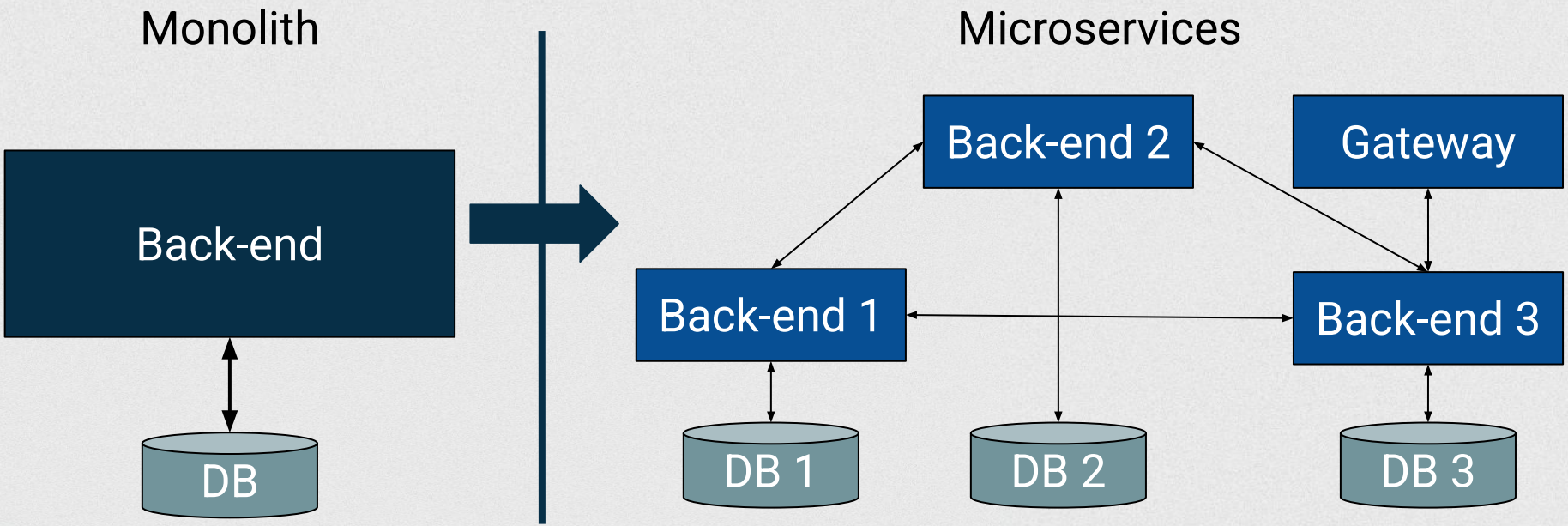
- Why are modern applications going to use event-driven microservices (EDM)?
- What is the difference between batch and stream processings and the Lambda architecture?
- What is event-driven architecture?

For the Lecture 2 (more practical questions):

- How does Kafka work?
- What is Kafka Stream?

1. Problems of Microservice Architecture

From Monolith to Microservices



What should we do, when the system becomes too complicated?
When we have to transmit the large amount of information between different parts of our application or our service?

Event-driven applied to microservices

1) Design Complexity:

   There are ***many different microservices instead of a monolith***. It is complicated to understand boundaries between microservices, their domain and the ***versioning.***

2) Operational Complexity:

   Having multiple services is the ***pressure on servers.***

3) Data Consistency:

   Classical microservices *need to* ***check*** *every time* ***multiple sources of data for consistency***.

4) Network Management:

   Having multiple services is the ***pressure on the network.***

Event-driven applied to microservices

5) Maintenance of Microservices:

There are ***many different microservices instead of a monolith.*** It might be ***multiple teams to maintain it.*** And they might be ***in different languages***.

6) Testing:

Developers need to test more, more cases, more services and you need to provide ***integration*** services.

7) Distributed monoliths:

Instead of distributed microservices ***it is risky to create another monolith.*** Synchronous point-to-point calls can mimic the monolith's boundaries.

8) Dependent scaling:

The ability ***to scale one service depends on the others***.

1.1 Real life challenges

Motivating example: e-commerce application

Where I worked:
A large e-commerce company, 10000+ developers.

My duties, which are connected to this course:

- Developed services on .NET Core to manage the logistics systems (an internal service for the maintaining of different delivery variations).
- Wrote jobs using Quartz.NET (a job scheduling library).
- Used Entity Framework Core/Dapper, GRPC/REST.
- Reviewed, refactored, tested, optimized and expanded code.
- Developed database structures (MS SQL, PostgreSQL).
- Released services to staging and production deployment environments.
- Used Apache Kafka.

Motivating example: our domain

Most important entities for an e-commerce company:
1) stores (where the production is located).
2) delivery variants (the ways, how the buyer obtains his ordered items) - our responsibility.
3) items (what we are delivering).

There were 10 microservices in our domain:
1) ***Delivery Variant System*** (the common delivery variants' data)**,** ***Warehouse System*** (the data about small warehouses)**,** ***TimeSlot System*** (the schedule of each delivery variant) like entity microservices.
2) ***Downloader Delivery Variants***; ***Uploader Delivery Variants***; ***Downloader Warehouses***; ***Uploader Warehouses***; ***Downloader Timeslots***; ***Uploader Timeslots*** - jobs for maintaining the data up to date and synchronize them.
3) ***Delivery Variant System Frontend*** for each user (our users were internal managers and technologists from our company).

Motivating example: our system

Internal microservices of a team:

1) Delivery Variant System

2) Warehouse System

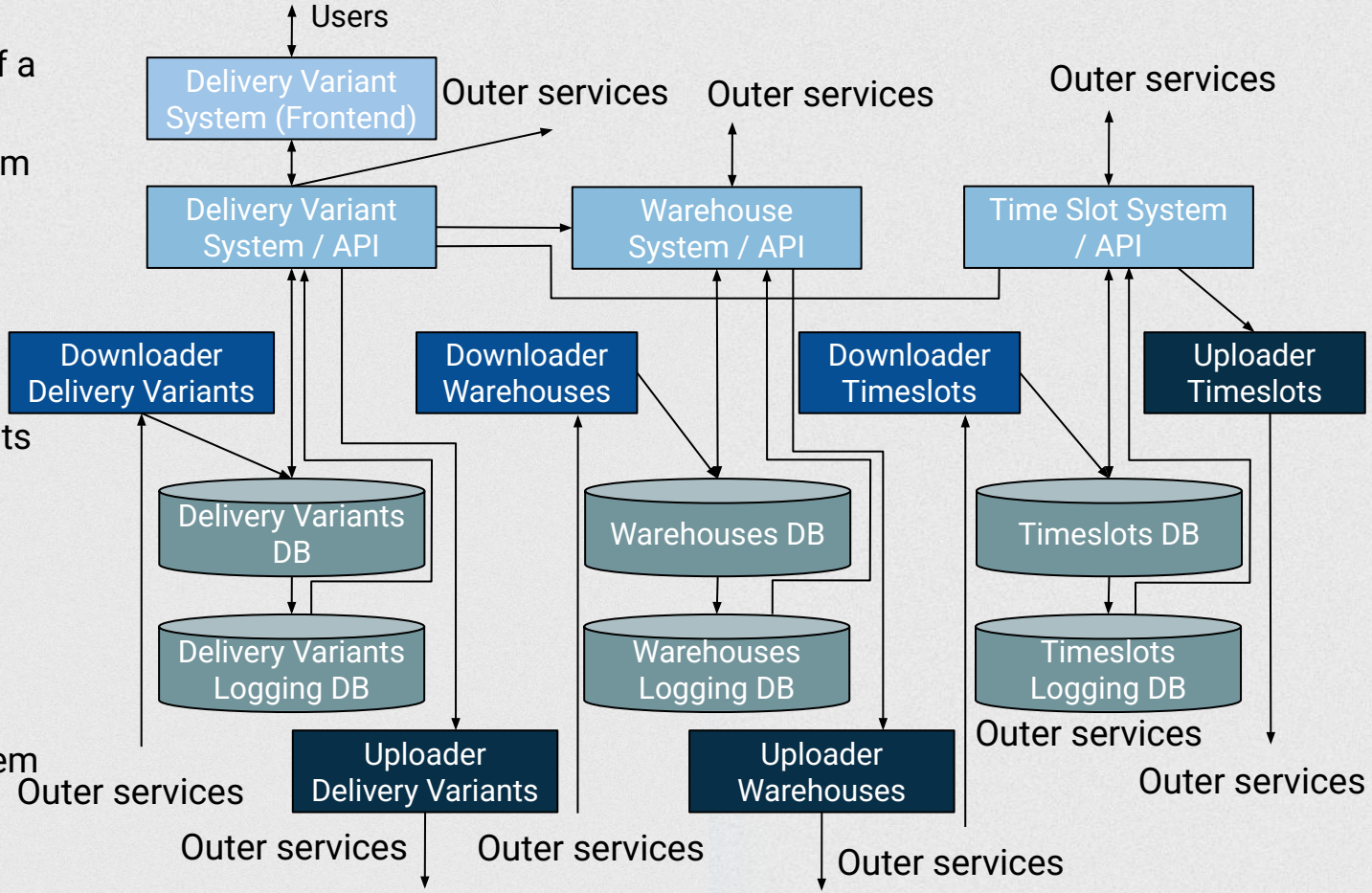
3) Time Slot System

4-5) Downloader / Uploader Delivery Variants

6-7) Downloader / Uploader Warehouses

8-9) Downloader / Uploader Timeslots

10) Delivery Varian System (Frontend)



Real life challenges. External microservices.

There are 4 other teams/microservices getting data from the service Delivery Variant System:

1) *Courier Delivery*
2) *Pick-up Points*
3) *Automatic Post Offices*
4) *Self-delivery*

There are 2 other teams/microservices getting data from Time Slot System:

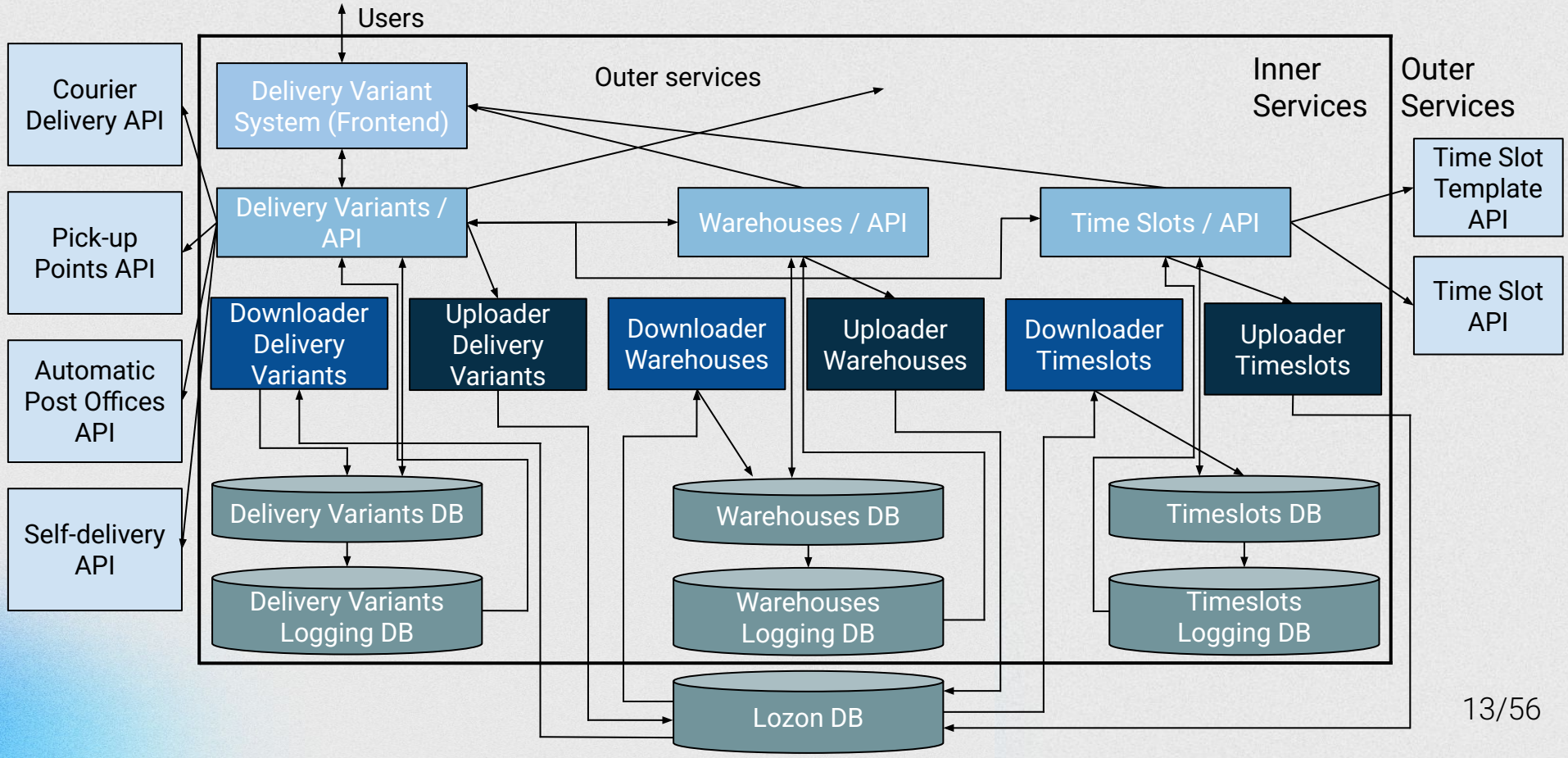
1) *Time Slot Template*
2) *Time Slot*

There is 1 monolithic database:

1) *Lozon*

Let’s add these external main microservices:

Real life challenges. External microservices.



Real life challenges. External microservices.

Add another two internal services using the same schema:

1) Downloader Delivery Variant Groups
2) Downloader Warehouse Groups

And some other important external services and teams, which we interact with these systems:

1-3) *Geo-polygon*, *Address* and *Geo Point* services:

The services for the providing of some geographical data for our delivery variants.

4) *ML service*:

The service for the analysis of our data.

5) *Downloader* service:

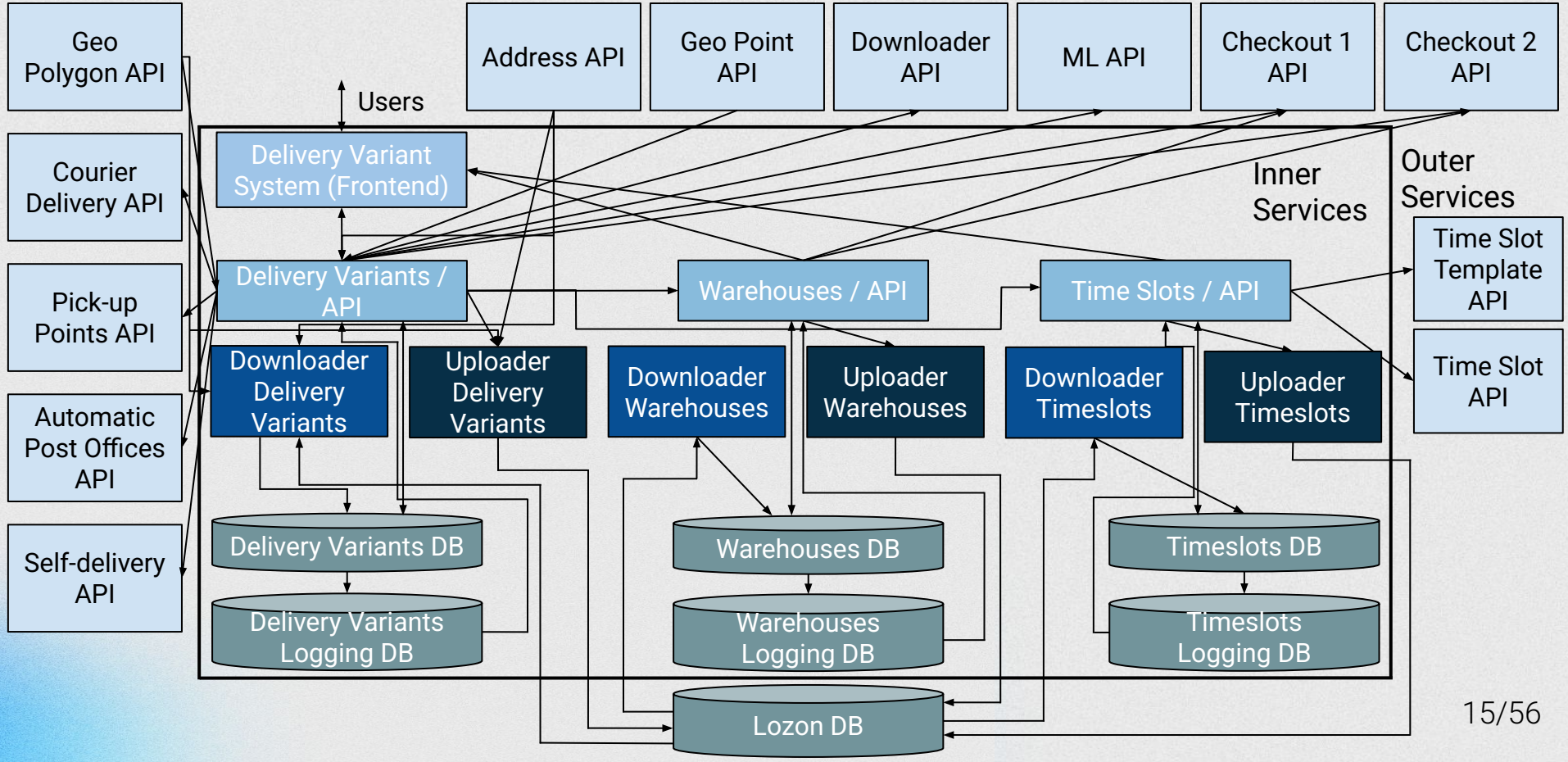
The service-connector between us and other internal services to the front-line services.

6-7) *Checkout* API #1 & #2:

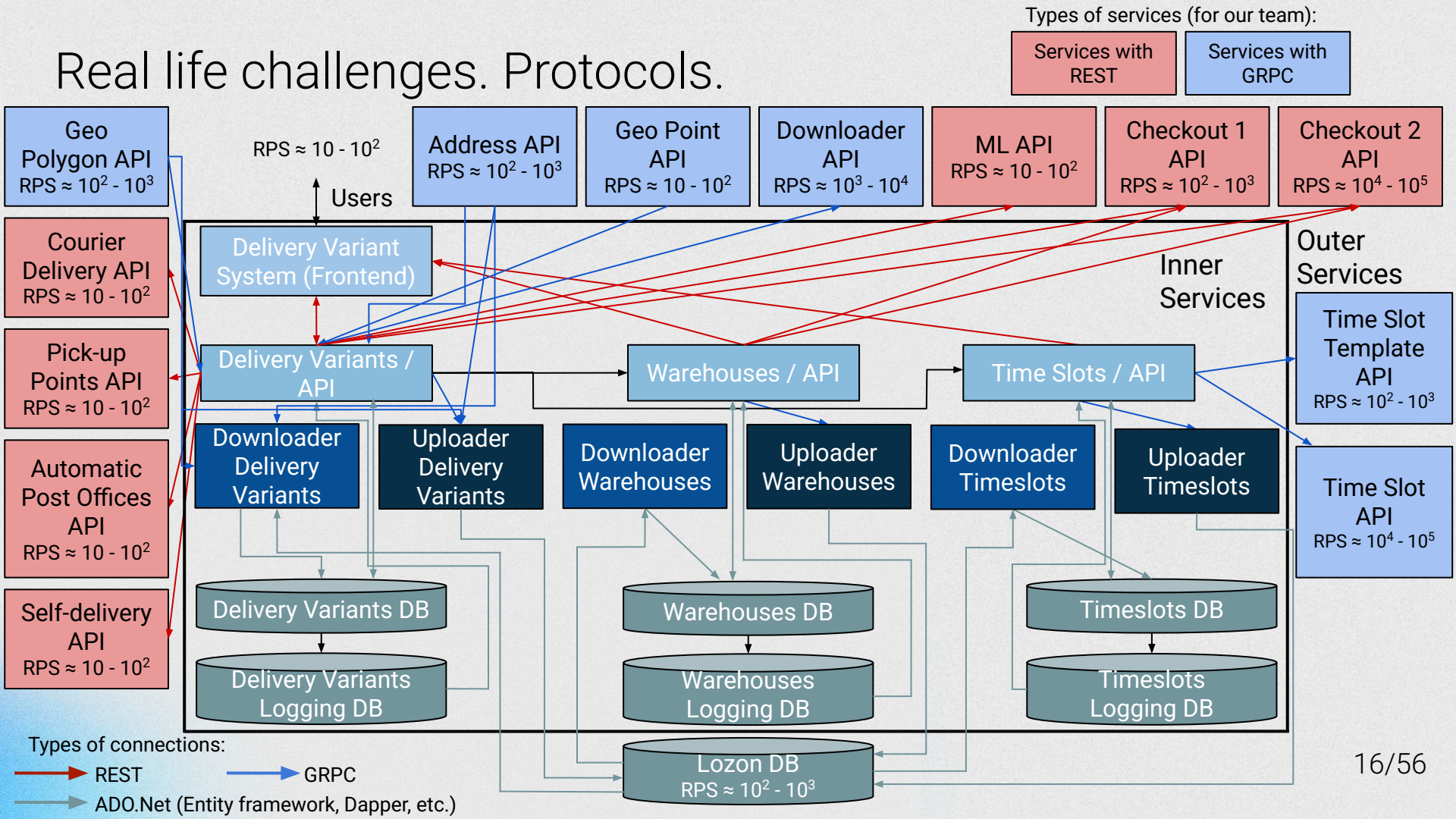
The services, working with items.

8) And 4 or 5 other teams.

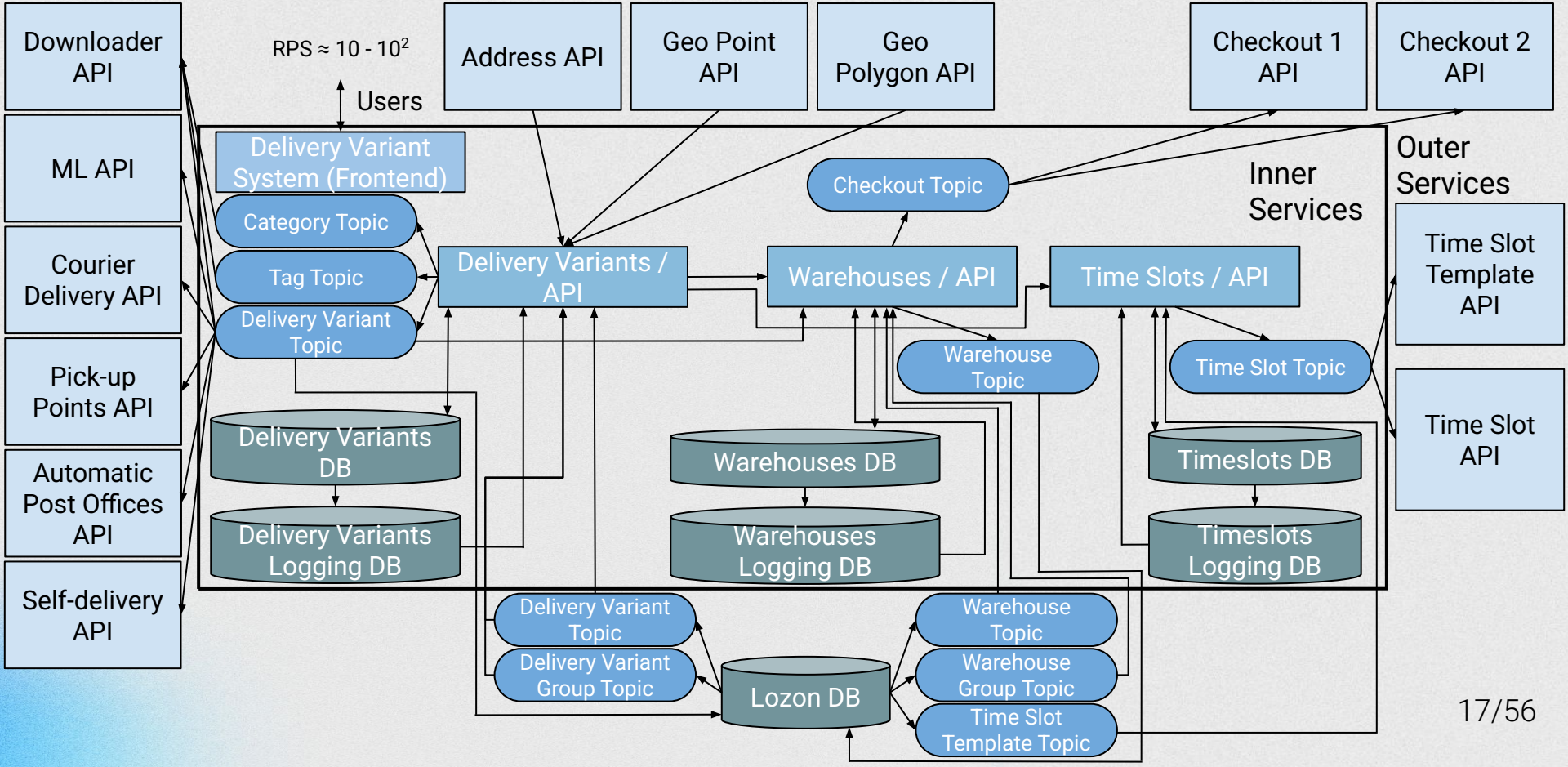
Real life challenges. External microservices.



Real life challenges. Protocols.



Real life challenges. Streaming.



Motivating example: observed benefits of EDM

1) Multiple downloaders/uploaders were deleted:

   The system was simplified.

2) Reactive changes:

   There is an asynchronous upsert of data.

3) Protocol unification:

   REST/GRPC/Dapper/etc. ways to provide/obtain data were diminished.

4) Decreasing computation pressure:

   High-demand resource requests were deleted.

5) Decreasing bandwidth:

   There are transmitting events instead of scheduled jobs with batches.

2. Processing

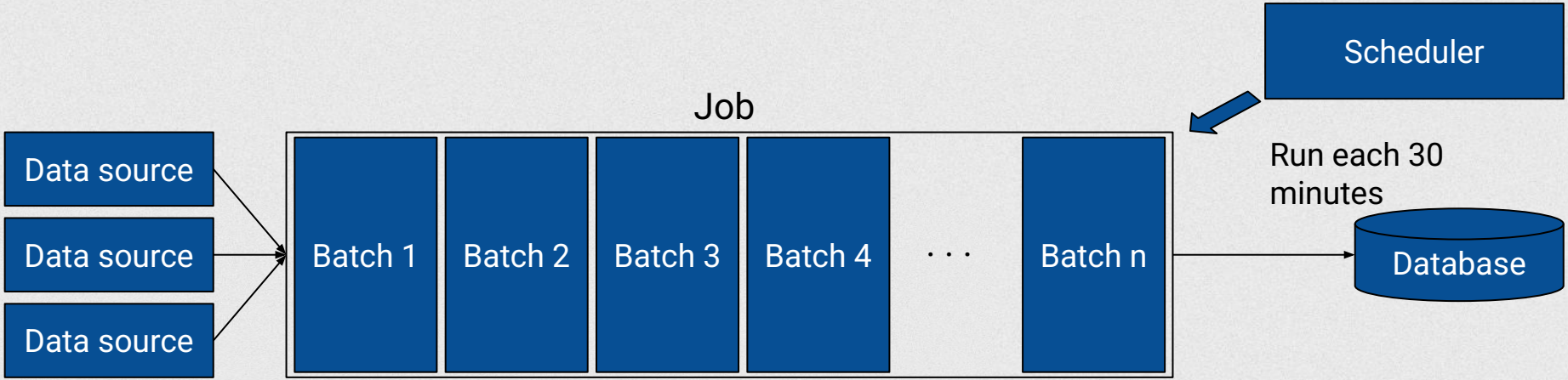
Types of processing

1) *Batch processing* - your program gathers information into batches and transmits them between services using jobs. Batches may automatically be run at scheduled times or being run contingent on the availability of computer resources.

2) *Stream processing* - your program focuses on real-time processing of **continuous streams of data** in motion.
Stream is a **sequence of data elements** made available over time.

3) *Lambda processing* - your program handles massive quantities of data by taking advantage of both batch- and stream-processing methods.

Batch processing



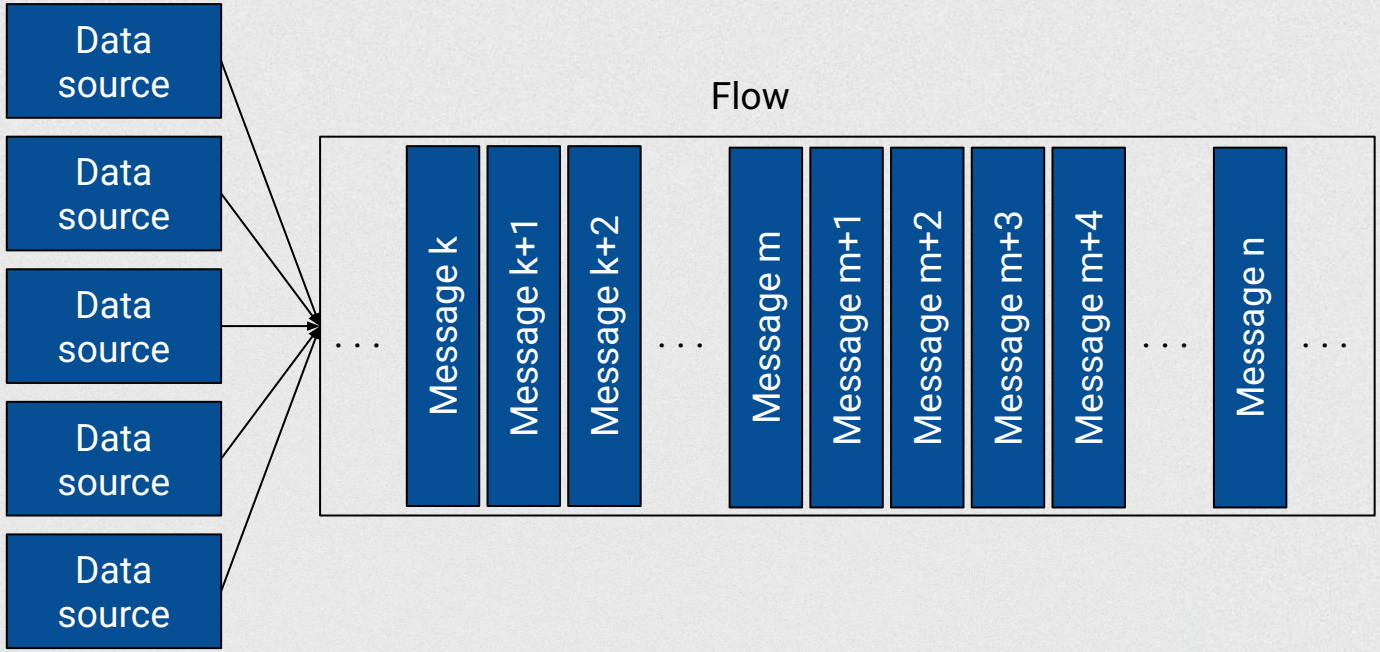
Batch processing is a method of running software programs called jobs in batches automatically.

Jobs gather information into the batches and transmit them between services.

Batch size:

- The number of lines from a file to load into a database.
- The number of messages to dequeue from a queue etc..

Stream processing



Stream processing focuses on the real-time processing of continuous streams of data in motion.

Stream processing. Characteristics.

Streams are:

- immutable - existing messages cannot be deleted.
- append-only - new messages can be added at the end of the stream.
- sequenced - there is the order by time of the creation of events.

Streams can be:

- partitioned - create a duplicate of the stream.
- divided - divide stream into different streams.
- a series of historical facts.

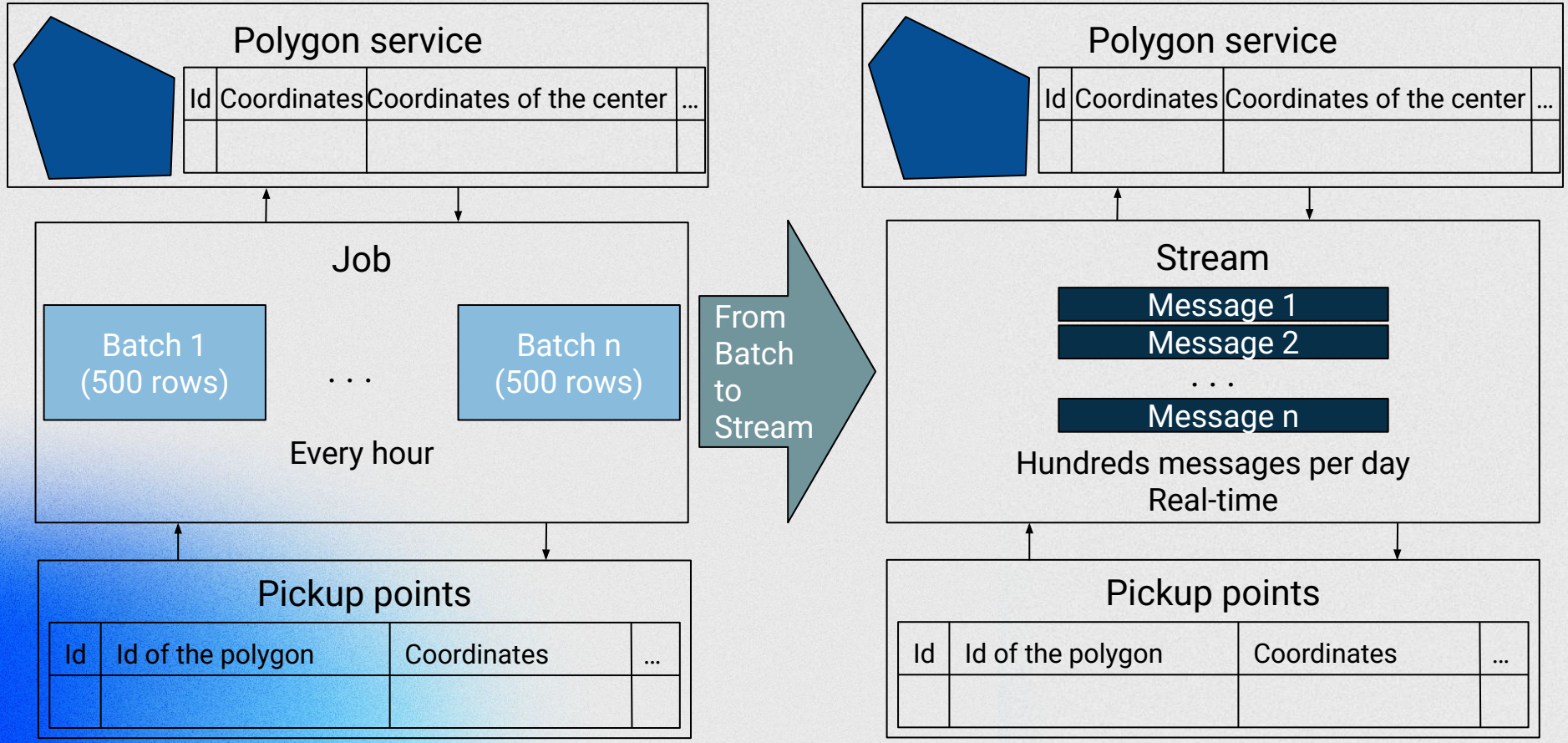
Examples:

- a sequence of financial transactions
- datasets for ML pipelines
- CRM systems

Filters (basic operations for Stream processing) produce another stream via:

- operate on one stream.
- operate multiple streams (e.g. an average).

Batch processing vs Stream Processing. Example.



Batch processing vs Stream processing

Batch processing is:

- easier for development:

Developers have to know less about different approaches and technologies.

- better for debugging:

Developers have to try to recreate sent message providing the same changes locally or using a developing environment to find the error.

- easier for testing:

It is easier to write unit tests

- more flexible for synchronization:

Developers could cease the running of jobs more easily.

Stream processing is:

- better for real-time issues:

Batch processing runs tasks every several minutes, but Stream processing send messages for each change and delay might be milliseconds.

- easier for maintaining:

It is easier to create another flow of data instead of the full rewriting of the job if you need to completely change your logic.

- more approachable for scaling:

It is complicated to use clusters for batches, because you need to synchronize them.

- better for resource consumption:

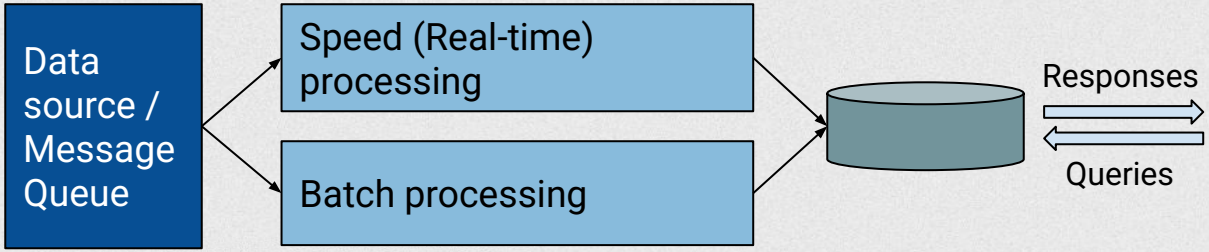
Developers need to transmit far more data with batch processing than with the stream processing because with the batch paradigm it needs to check the changes for all our data

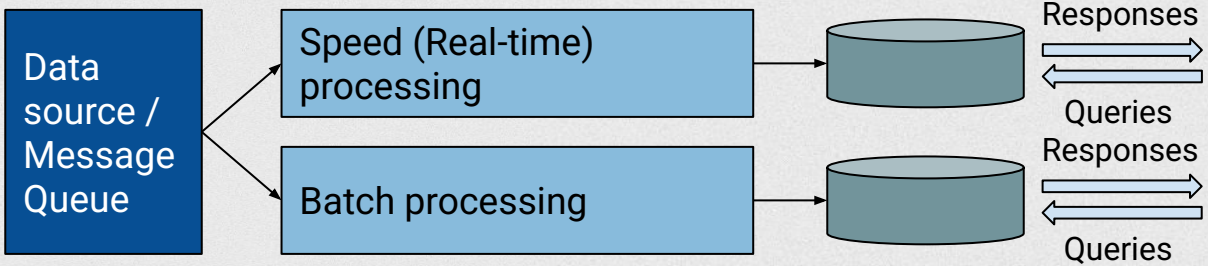
2.3 Lambda architecture

Source: https://www.databricks.com/glossary/lambda-architecture

Lambda architecture

- Designed to handle massive quantities of data.
- It takes advantage of both batch and stream-processing methods.
- Stream processing complements batch-processing, but it doesn't allow you to get the full information, only the important one.





Lambda architecture

Example for the entire system are:

- Apache Druid etc..

For different systems are:

- For real-time processing might be:
  - Apache Cassandra
  - MongoDB etc..
- For batch-layer output are:
  - Apache Impala
  - SAP HANA etc..

Lambda is used by:

- Netflix
- Paypal
- Pinterest
- Lyft
- X (ex-Twitter) etc..

Lambda architecture. Benefits.

- Low latency

The speed layer uses stream processing to transmit data in the near real-time, reducing the latency.

- Data consistency

Even little possible risks of inconsistency data from the stream because of the network failures could be checked, found and overlaid by the batch, when it is started. Batch processing is an additional control mechanism.

- Scalability

Lambda can use the multiple node decision for both streams and batches. Other nodes could share the computational power.

- Fault tolerance

Lambda can use the multiple node decision for both streams and batches. When one node fails, other nodes will distribute the work of the failed node.

Lambda architecture. Criticism and alternatives.

Drawbacks:

- Can be complicated (different code base for batching and streaming).
- Limited influence.

Kappa architecture:
Use the single code base for stream and historical data. It leans to stream processing. An example is Apache Samza.

Delta architecture:
Additional table storage layer to store both stream and batch data. Example is Data Lake from Databricks.

3. Event-driven architecture

Source: “Building Event-Driven Microservices: Leveraging Organizational Data at Scale” Adam Bellemare, O'Reilly Media, 2020

Event-driven architecture

- Anything that happens at a point in time is an **event** (i.e. the submission of an invoice, log line emitted by a web server, etc.).
- Event-driven architecture uses **events** to trigger and communicate between decoupled services.
- The events could carry and change **the information**, or could change **the state** using notification.

Components:

- Event-producer (to publish events).
- Event-router (to filter events).
- Event-consumer (to obtain events).

It allows you to create/update/scale producer services and consumer services **independently**.

What Are Event-Driven Microservices?

- Event-based architecture **is not new**, but the need for handling big data sets, at scale and in real time, **is new**.
- The events are not destroyed upon consumption and remain available for other consumers to read as they require.
- The services are small (no more than two weeks to write / should be able to fit within one’s own head).

Event-Driven Concepts

- ***Events are the basis of communication*.**
All shareable data is published to a set of event streams. Nearly anything can be communicated as an event. Events *are* the data.

- ***Event streams provide the single source of truth*.**
Each event in a stream is a statement of fact.

- ***Consumers perform their own modeling and querying*.**
Data access and modeling requirements are completely shifted to the consumer. The consumer is responsible for any mixing of data from multiple event stream, queries, etc..

- ***Data communication is improved across the organization*.**
Applications can access data that would otherwise be laborious to obtain via point-to-point connections.

- ***Accessible data supports business communication changes*.**
Event streams contain core events that are central to the operation of the business. Other teams know where they can find these data.

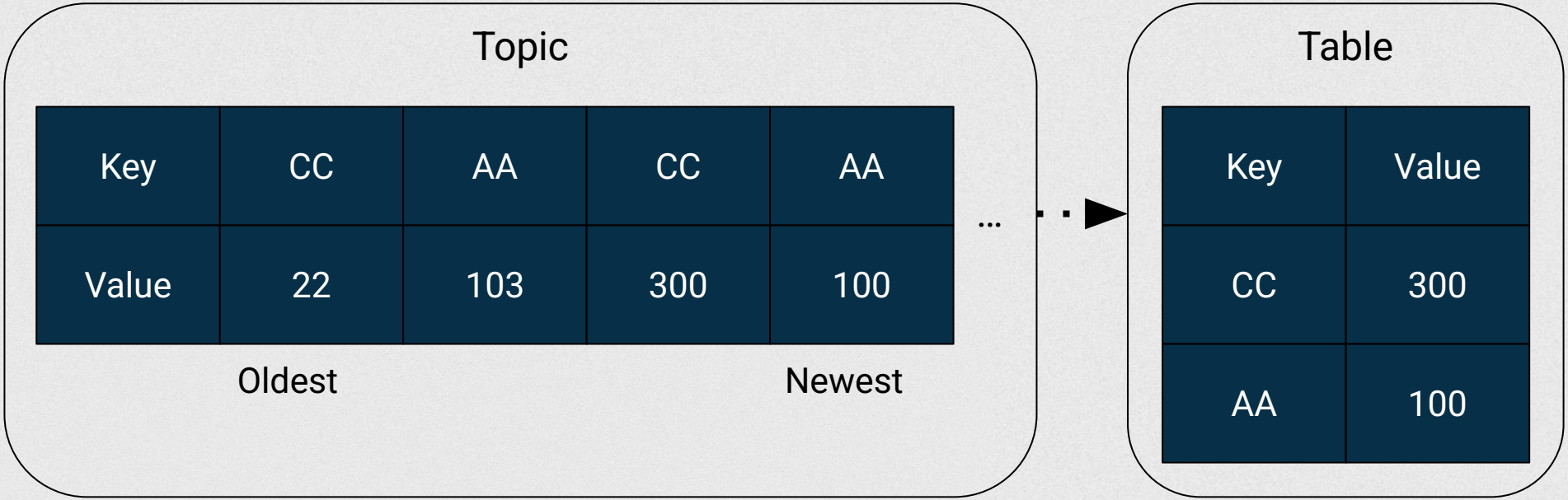
The Contents of an Event

An event is recording of what happened. But unlike the logs, events are the single source of truth.

Events typically have a key/value format. The value stores the details of the event, while the key is used for identification purposes/routing/aggregation etc..

Type of the event	Key	Value
Unkeyed event - a singular statement of fact	N/A	ISBN: 112233, Timestamp: 1538913600
Entity event - unique thing, represents entity.	ISBN: 112233	Author: authorName
Keyed event - unique thing, does not represent an entity.	ISBN: 112233	UserId: A537FE

Materializing State from Entity Events



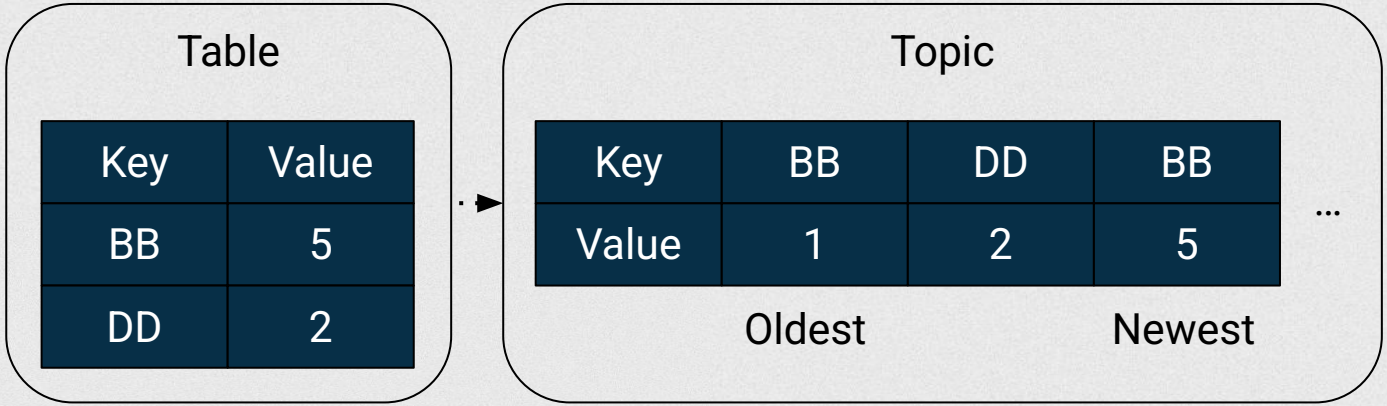
You can ***materialize*** a stateful table by applying entity events. Each entity event is upserted into the key/value table.

Topic is a category name to which records are stored and published.

Materializing State from Entity Events

Commands executed

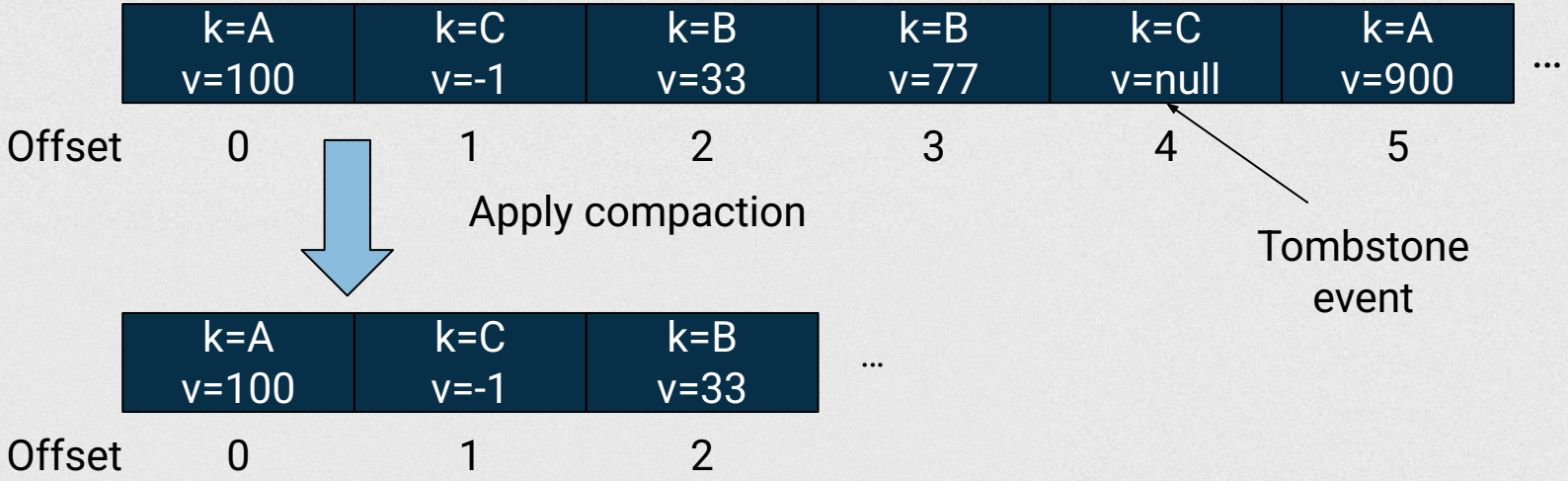
Upsert (BB, 1)
Upsert (DD, 2)
Upsert (BB, 5)
...



Conversely, you can convert a table into a stream of entity events by publishing each update to the event stream.

This is known as the ***table-stream duality***.

Materializing State from Entity Events



The deletion of a keyed event is handled by producing a ***tombstone*** (a keyed event with its value set to null).

Append-only immutable logs may grow indefinitely unless they are compacted. ***Compaction*** is performed by the event broker to reduce the size of its internal logs by retaining only the most recent event for a given key.

Schema format evolution

Schema defines the structure of ***the data format***. Shema evolution enables producers to update their service's output format while allowing consumers to continue consuming the events uninterrupted.

Compatibility types:

- ***Forward compatibility*** - allows for data produced with a newer schema to be read as though it were produced with an older schema. The consumer is required to update its copy of the schema and to get access to the new fields.
- ***Backward compatibility*** - allows for data produced with an older schema to be read as though it were produced with a newer schema.
  Scenarios:
  - the consumer is updated before the producer.
  - the product deployed on customer hardware (e.g. cell phone applications).
  - reprocess data by the consumer, that was produced with an older schema version.
- ***Full compatibility*** - the union of forward and backward compatibility.

These approaches are applied not only for event-driven approaches, but for each and every deployment in microservice architectures and teams need to follow these rules.

Event-Driven Microservices. Benefits.

Benefits over the regular microservices:

- *Granularity* - services can be easily rewritten according to the business requirements.
- *Improve scalability* - scaled up and down as needed.
- *Technological flexibility* - using pioneering technologies / the most appropriate languages.
- *Business requirement flexibility* - more convenient for business changes;
- *Loosely coupling* - doesn't need to use APIs.
- *Improve continuous delivery support* - easy to ship a small microservices and roll them back (if needed).
- *High testability* - easier to mock / to cover the code.

4. Brokers

Brokers. Overview.

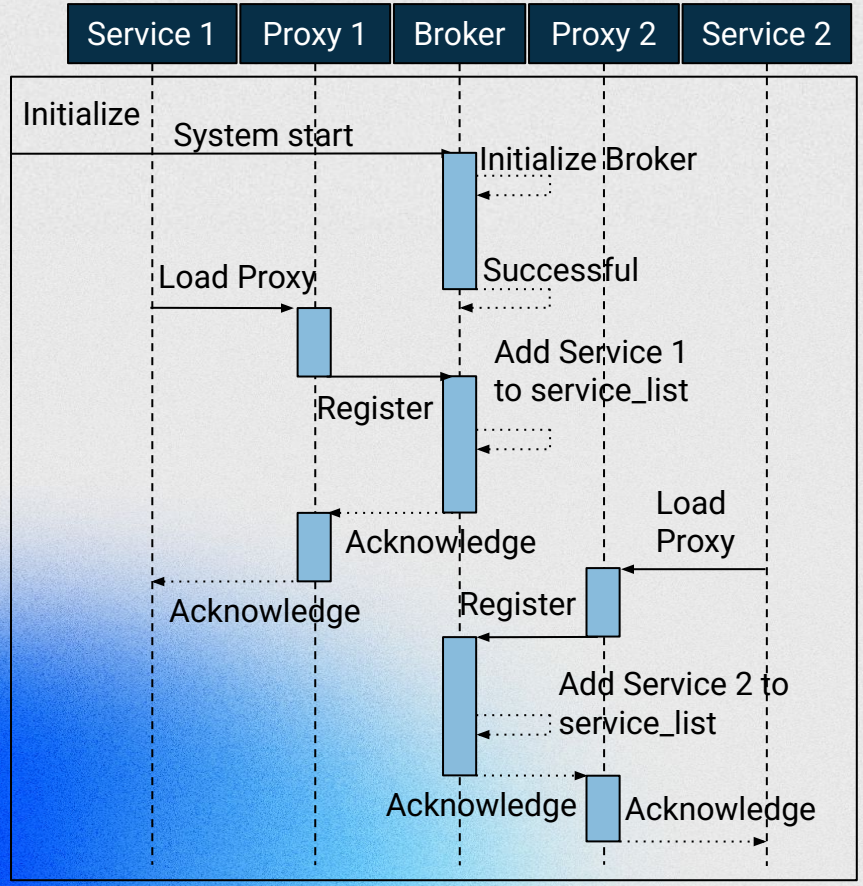
It is a program module and architectural pattern translating messages from the formal messaging protocol of the **sender** to the formal messaging protocol of the ***receiver***.

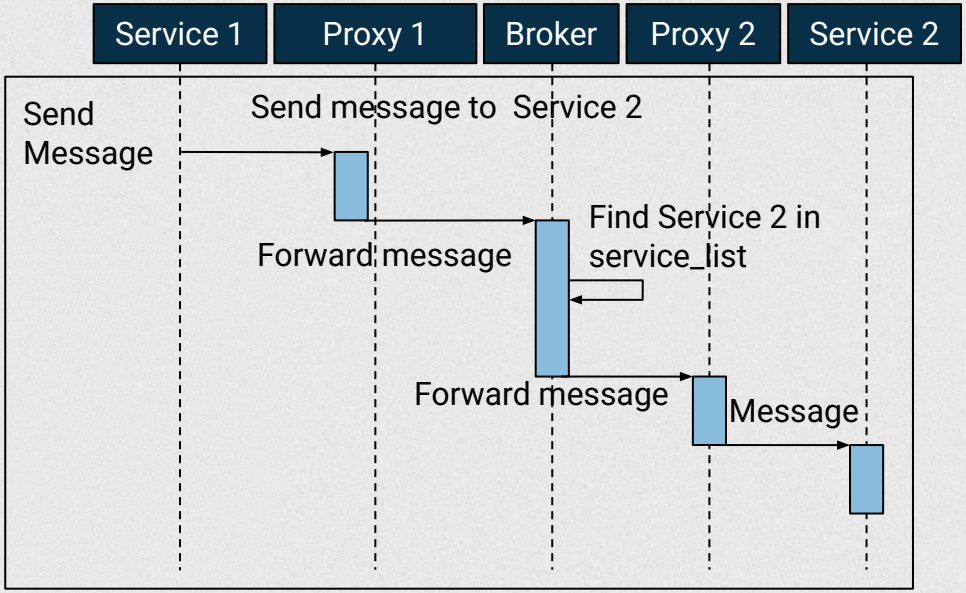
It provides the **validation**, **transformation** and **routing** of messages and implements service decoupling.

Examples of actions:

- Route messages to one or more destinations. Provide content and topic-based message routing using the publish–subscribe pattern.
- Transform messages to an alternative representation.
- Respond to events or errors.

Brokers. The principle of operation. Example.





Types of brokers. Features of event broker.

2 types of brokers:

- Message broker
- Event broker

Features of event brokers:

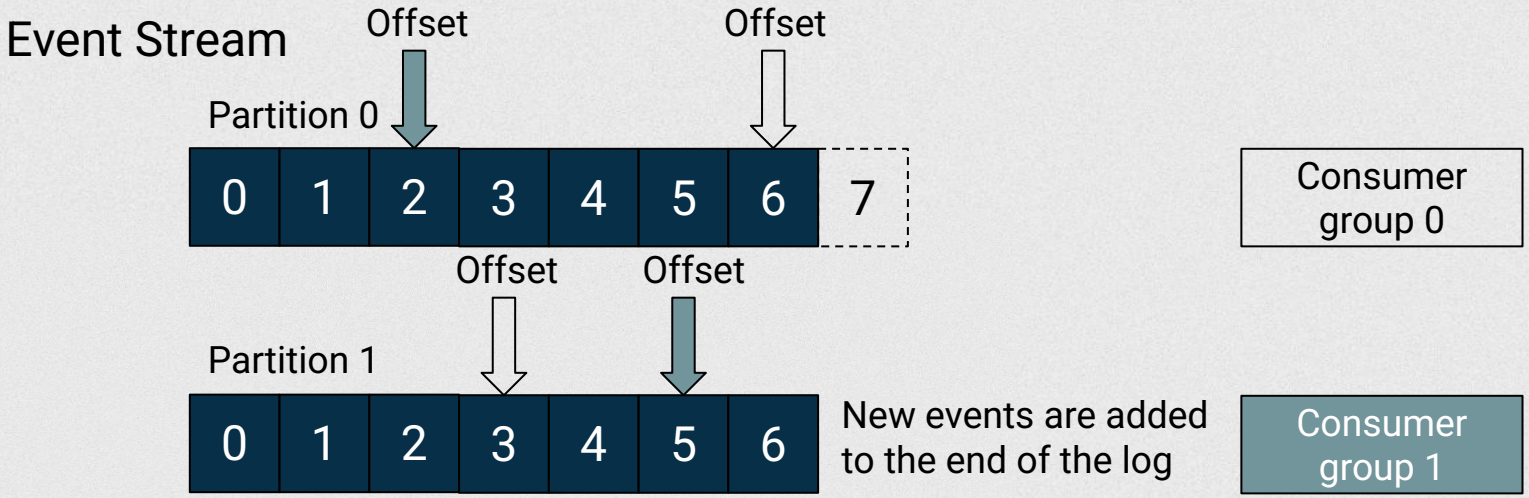
- *Scalability:*
  An event broker can be added/removed to increase/decrease the cluster's production / consumption / data storage capacity.
- *Durability:*
  Event data is replicated between nodes (event brokers). The cluster can preserve and continue serving data when a broker fails.
- *High availability:*
  A cluster of event broker nodes enables clients to connect to other nodes in the case of a broker failure.
- *High-performance:*
  Multiple broker nodes share the production and consumption load. **Each broker is a highly performant separated server.**

Event Storage and Serving requirements for brokers

Minimal requirements of the underlying storage of the data by broker:

- *Partitioning:*
  Event stream can be partitioned into individual substreams to process each substream in parallel.
- *Strict ordering:*
  Data in an event stream partition is strictly ordered.
- *Immutability:*
  All event data is completely immutable once published.
- *Indexing:*
  Event are assigned an index when written to the event stream.
- *Infinite retention:*
  Event streams must be able to retain events for a certain period of time.
- *Replayability:*
  Event streams must be replyable, such that any consumer can read whatever data it requires.

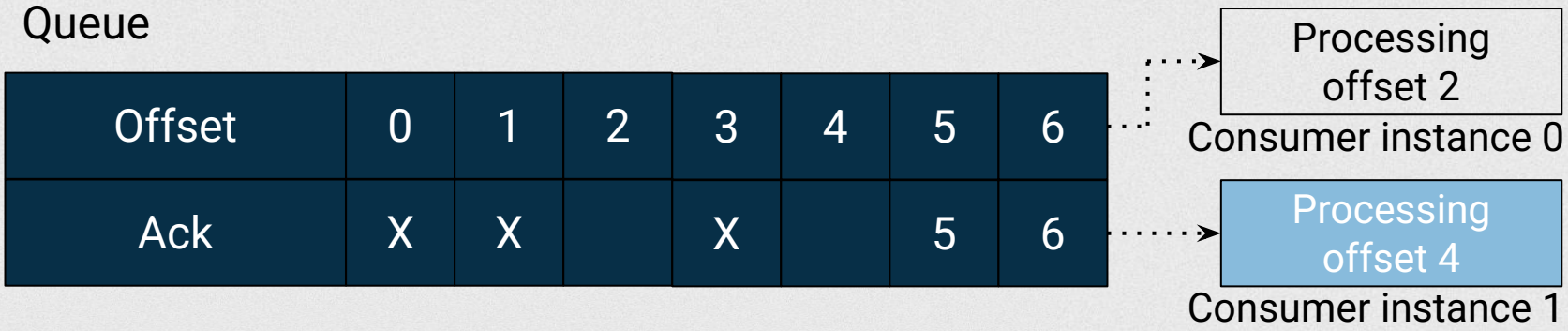
Consuming from the Immutable Log as an Event Stream



Each consumer is responsible for updating its own pointers to previously read indices (*offsets*) within the event stream.

The *consumer group* (multiple joined consumers) allows for multiple consumers to be viewed as the same logical entity and can be leveraged for horizontal scaling of message consumption. The number of active consumer instances in the group is limited to the number of partitions in the event stream.

Consuming from the Immutable Log as an a Queue



In queue-based consumption, each event is consumed by one and only once microservice instance. The event is marked as “consumed” by the event porter and is no longer provided to any other consumer.

Event Brokers vs. Message Brokers

Event Brokers

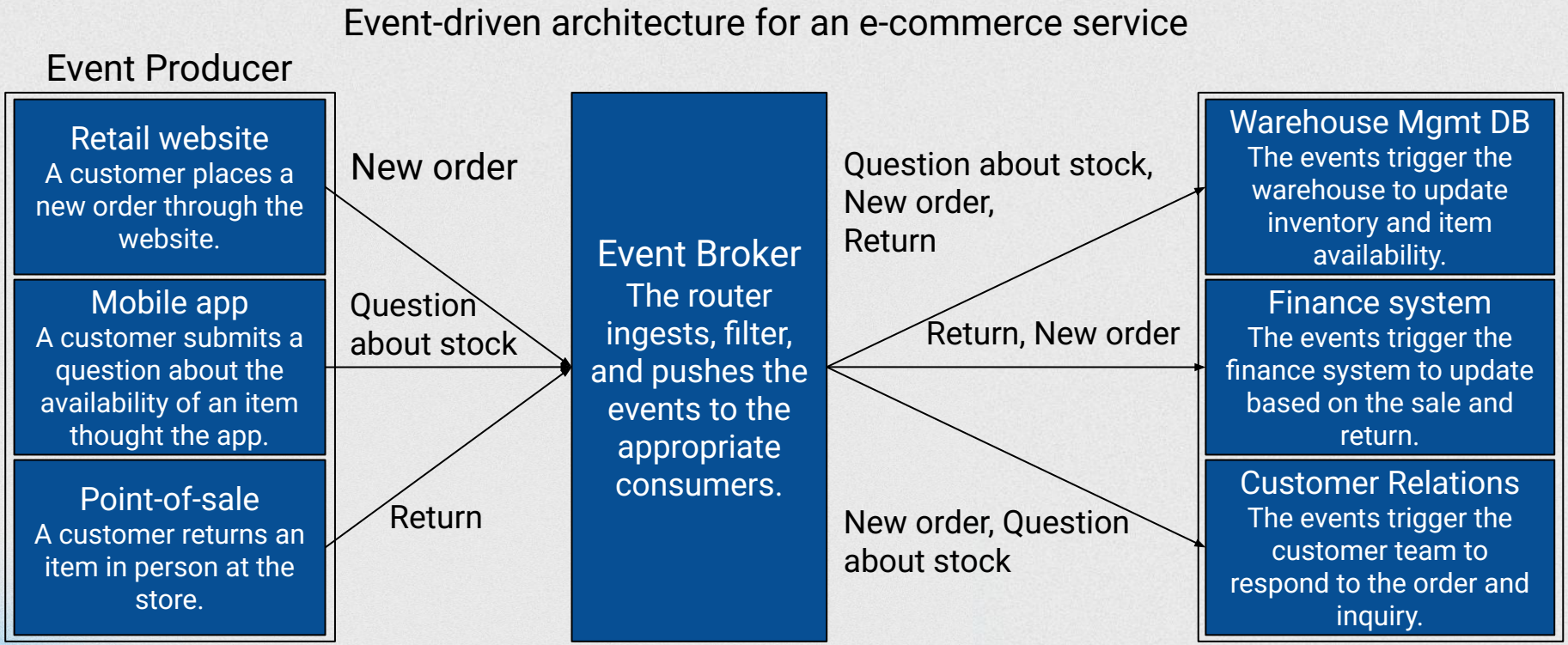
- Provides an ordered log of facts.
- Each independent consumer can access all required events.
- The event broker retains events for as long as needed.

Message Brokers

- Provides only queues of messages.
- Each consumer is unable to obtain a full copy of all events.
- Message broker deletes events after acknowledgement.

The consumer can pick up and reprocess events from anywhere in the log at any time. This pattern is essential for EDM, but it is not available with message brokers.

Event broker. Example.



Event-driven architecture. Examples.

- *Fanout and parallel processing:*

  If you have a lot of different services, you don’t need to write custom code to push data to each service.

- *Resource state monitoring and alerting:*

  You can use event-driven architecture to monitor and receive alerts on any anomalies, changes and updates. It could include all resources, like storage buckets, database tables, serverless functions, compute nodes and more.

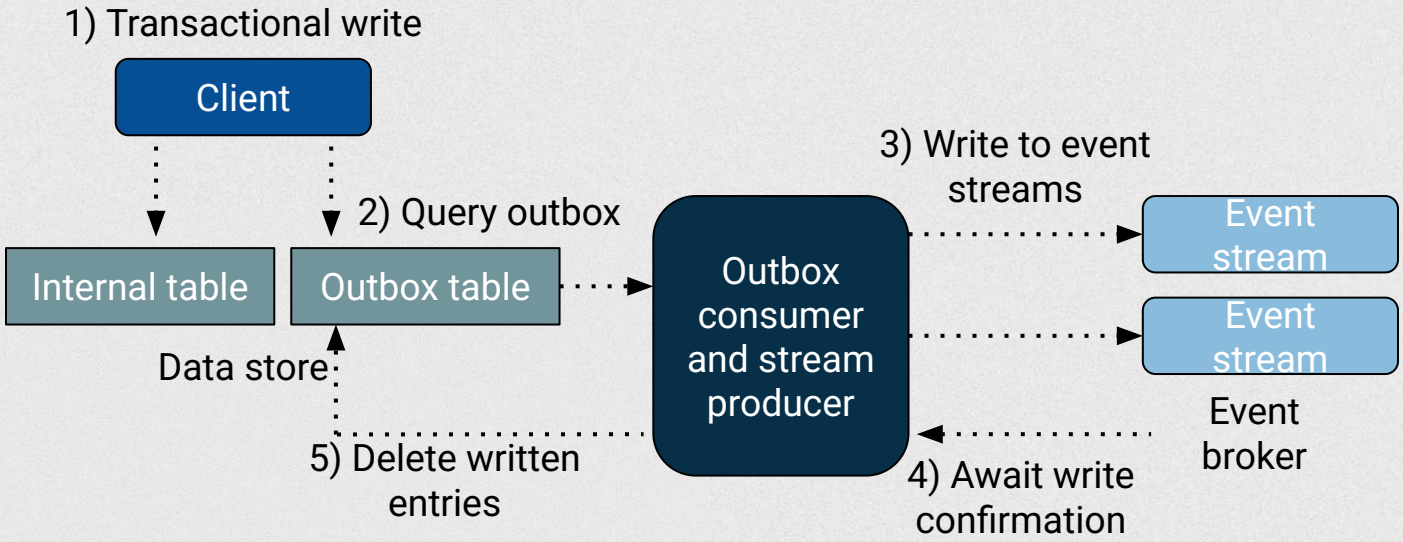
- *Integration of heterogeneous systems*

  It is an example from my job, when we had two different stacks, many teams wrote different protocols and languages and event-driven architecture helps to connect these stacks without coupling.

5. Data Liberation

Liberation Data Using Outbox Tables

Outbox Table: Stores events alongside transactional data for consistency.
Polling Mechanism: Background processes publish entries from the outbox to other systems.



Fault Tolerance: Data remains in the outbox until successfully processed.
Eventual Consistency: Ensures data consistency across services over time.

Liberation Data Using Outbox Tables (Example)

Upsert command

Id int	Name Text	PostalCode Int
1	Adam	1170

Upsert at time T

1) Upsert to internal table

User internal table

Id int	Name Text	PostalCode Int
1	Adam	1170

If the trigger fails, the update to user internal table fails

2 & 4) Trigger executes as part of transaction

3) Upsert to internal table

1	Adam	45601

Upsert at time T+1

User outbox table

Id int	Name Text	PostalCode Int	UpdatedAt Time	SeqId Int
1	Adam	1170	T	1
1	Adam	45601	T+1	2

User output event stream schema for validation:
Key = {ID:Integer}
Value = {Name:String, PostalCode:Integer}

5) Poll data

6) Perform validation

7) Compose events and publish

Schema validation	Event publisher

Outbox consumer and stream producer

If the validation fails, the event cannot be published without manual intervention

User event stream

Record Header	Timestamp: T+1	Timestamp: T
Key	Id: 1	Id: 1
Value	Name: Adam PostalCode: 45601	Name: Adam PostalCode: 1170
	Offset: 2	Offset: 1

OUTLINE

- Problems of microservice Architecture:
  - Real life challenges
- Processing:
  - Batch processing
  - Stream processing
  - Lambda architecture
- Event-driven architecture
- Brokers
- Data Liberation

NEXT LECTURE

- Kafka:
  - Fundamentals
  - Consumer offset semantics
- Kafka Streams:
  - Basic concept
  - Stream SQL
  - ksqlDB
- RabbitMQ
  - Kafka vs RabbitMQ
- Assignment

20.11.25

Stream Processing (Message Brokers and Kafka) Part 1

University of Vienna / Faculty of Computer Science

Andrey Nagiyev BSc MSc andrey.nagiyev@univie.ac.at